

Knowledge Distillation

March, 2025

Table of Contents

- 1 Distilling the Knowledge in a Neural Network
- 2 Knowledge Distillation of Large Language Models
 - Introduction
 - Knowledge Distillation Algorithms
 - Skill Distillation
 - Domain-Specified Vertical Distillation
- 3 Examples: DistilBERT and TinyBERT
- 4 References

Motivation

- A very simple way to improve the performance of almost any machine learning algorithm is to train many different models on the same data and then to average their predictions.
- Unfortunately, making predictions using a whole ensemble of models is cumbersome and may be too computationally expensive to allow deployment to a large number of users, especially if the individual models are large neural nets.

What is knowledge

- A **conceptual block** that may have prevented more investigation of this very promising approach is that we tend to identify the knowledge in a trained model with the **learned parameter values** and this makes it hard to see how we can change the form of the model but keep the same knowledge.
- A more abstract view of the knowledge, that frees it from any particular instantiation, is that it is a **learned mapping from input vectors to output vectors**.

What is knowledge (Cont'd)

- For cumbersome models that learn to discriminate between a large number of classes, the normal training objective is to maximize the average log probability of the correct answer, but a side-effect of the learning is that the trained model assigns probabilities to all of the incorrect answers and even when these probabilities are very small, some of them are much larger than others.
- The relative probabilities of incorrect answers tell us a lot about how the cumbersome model tends to generalize. An image of a BMW, for example, may only have a very small chance of being mistaken for a garbage truck, but that mistake is still many times more probable than mistaking it for a carrot.

Soft-target training

- An obvious way to transfer the generalization ability of the cumbersome model to a small model is to use the class probabilities produced by the cumbersome model as “soft targets” for training the small model.
- When the cumbersome model is a large ensemble of simpler models, we can use an arithmetic or geometric mean of their individual predictive distributions as the soft targets. When the soft targets have high entropy, they provide much more information per training case than hard targets and much less variance in the gradient between training cases, so the small model can often be trained on much less data than the original cumbersome model and using a much higher learning rate.

Temperature and distillation

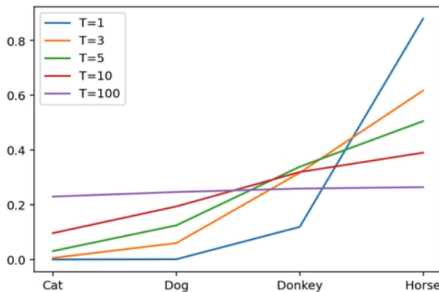
- For tasks like MNIST in which the cumbersome model almost always produces the correct answer with very high confidence, much of the information about the learned function resides in the ratios of very small probabilities in the soft targets.
- For example, one version of a 2 may be given a probability of 10^{-6} of being a 3 and 10^{-9} of being a 7 whereas for another version it may be the other way around. This is valuable information that defines a rich similarity structure over the data (i. e. it says which 2's look like 3's and which look like 7's) but it has very little influence on the cross-entropy cost function during the transfer stage because the probabilities are so close to zero.

Temperature and distillation (Cont'd)

Neural networks typically produce class probabilities by using a "softmax" output layer that converts the logit, z_i , computed for each class into a probability, q_i , by comparing z_i with the other logits.

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

where T is a temperature that is normally set to 1. Using a higher value for T produces a softer probability distribution over classes.



Temperature and distillation (Cont'd)

- In the simplest form of distillation, knowledge is transferred to the distilled model by training it on a transfer set and using a soft target distribution for each case in the transfer set that is produced by using the cumbersome model with a high temperature in its softmax.
- The same high temperature is used when training the distilled model, but after it has been trained it uses a temperature of 1.

Double target

We have found that using the original training set works well, especially if we add a small term to the objective function that encourages the small model to predict the true targets as well as matching the soft targets provided by the cumbersome model.

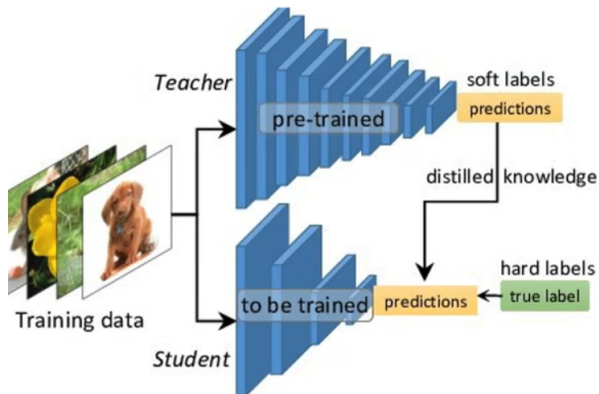


Table of Contents

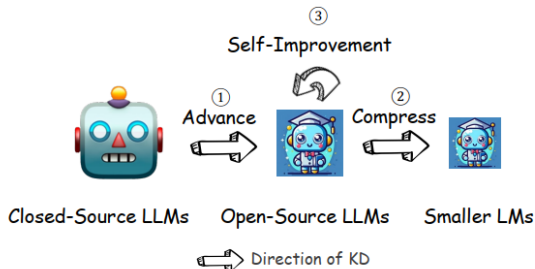
- 1 Distilling the Knowledge in a Neural Network
- 2 Knowledge Distillation of Large Language Models
 - Introduction
 - Knowledge Distillation Algorithms
 - Skill Distillation
 - Domain-Specified Vertical Distillation
- 3 Examples: DistilBERT and TinyBERT
- 4 References

Table of Contents

- 1 Distilling the Knowledge in a Neural Network
- 2 Knowledge Distillation of Large Language Models
 - Introduction
 - Knowledge Distillation Algorithms
 - Skill Distillation
 - Domain-Specified Vertical Distillation
- 3 Examples: DistilBERT and TinyBERT
- 4 References

Background

- In the era of Large Language Models (LLMs), Knowledge Distillation (KD) emerges as a pivotal methodology for **transferring advanced capabilities from leading proprietary LLMs**, such as GPT-4, to their open-source counterparts like LLaMA and Mistral.
- Additionally, as open-source LLMs flourish, KD plays a crucial role in both **compressing these models**, and **facilitating their self-improvement** by employing themselves as teachers.



Distillation (traditional recipe)

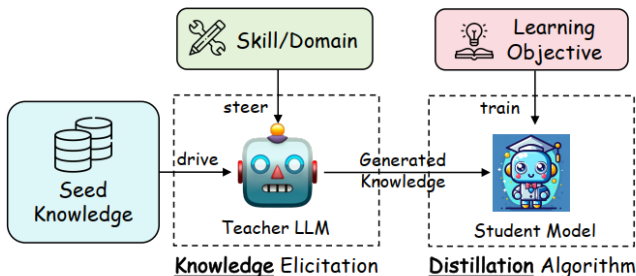
- The concept of knowledge distillation in the field of AI and deep learning (DL) refers to the process of transferring knowledge from a large, complex model (teacher) to a smaller, more efficient model (student).
- The focus was predominantly on ad-hoc neural architecture selection and training objectives tailored for single tasks.
- These earlier methods involved training a smaller student network to mimic the output of a larger teacher network, often through techniques like soft target training, where the student learns from the softened softmax output of the teacher.

Distillation (LLM era)

- The current era of knowledge distillation in LLMs shifts the focus from mere **architecture compression** to **knowledge elicitation and transfer**.
- This paradigm change is largely due to the expansive and deep-seated knowledge that LLMs like GPT-4 and Gemini possess. And the inaccessible parameters of LLMs make it hard to compress them by using pruning or quantization techniques.
- The key to this modern approach lies in heuristic and carefully designed **prompts**, which are used to elicit specific knowledge or capabilities from the LLMs.
- Furthermore, this era of knowledge distillation also emphasizes the transfer of more abstract qualities such as reasoning patterns, preference alignment, and value alignment.

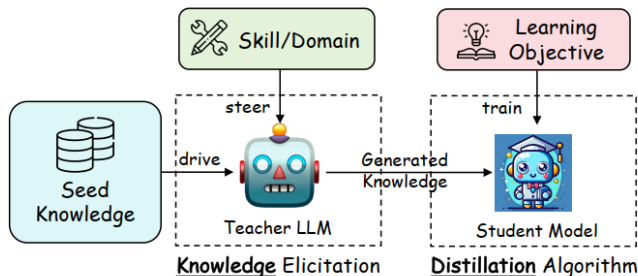
Distillation pipeline in LLM era

- **I. Target Skill or Domain Steering Teacher LLM.** The first stage involves directing the teacher LLM towards a specific target skill or domain. This is achieved through carefully crafted instructions or templates that guide the LLM's focus. These instructions are designed to elicit responses that demonstrate the LLM's proficiency in a particular area or a skill.



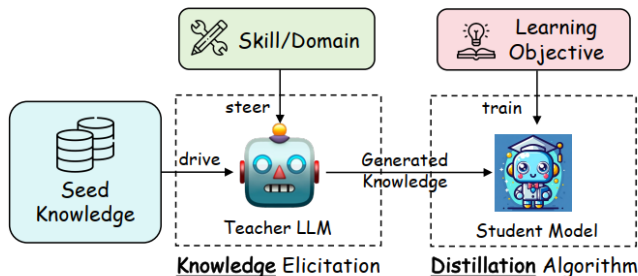
Distillation pipeline in LLM era

- **II. Seed Knowledge as Input.** Once the target area is defined, the next step is to feed the teacher LLM with seed knowledge. This seed knowledge typically comprises a small dataset or specific data clues relevant to the elicit skill or domain knowledge from the teacher LLM. It acts as a catalyst, prompting the teacher LLM to generate more elaborate and detailed outputs based on this initial information.



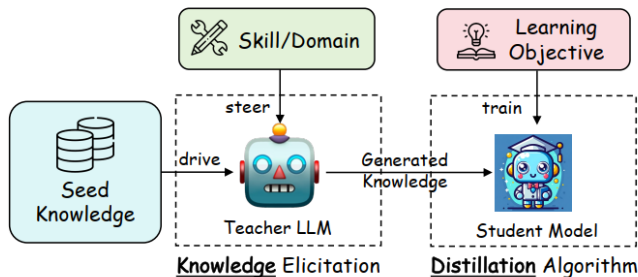
Distillation pipeline in LLM era

- **III. Generation of Distillation Knowledge.** In response to the seed knowledge and steering instructions, the teacher LLM generates knowledge examples. These examples are predominantly in the form of question-and-answer (QA) dialogues or narrative explanations, aligning with the natural language processing/understanding capabilities of the LLM.



Distillation pipeline in LLM era

- **IV. Training the Student Model with a Specific Learning Objective.** The final stage involves the utilization of the generated knowledge examples to train the student model. This training is guided by a loss function that aligns with the learning objectives.



Distillation pipeline in LLM era

In essential, the above four stages can be abstracted as two formulations. The first formulation represents the process of eliciting knowledge:

$$\mathcal{D}_I^{(\text{kd})} = \{\text{Parse}(o, s) \mid o \sim p_T(o \mid I \oplus s), \forall s \sim \mathcal{S}\}$$

where $\oplus$ denotes fusing two pieces of text, I denotes an instruction or a template for a task, skill, or domain to steer the LLM and elicit knowledge, $s \sim \mathcal{S}$ denotes an example of the seed knowledge, upon which the LLM can explore to generate novel knowledge, $\text{Parse}(o, s)$ stands for to parse the distillation example (e.g., (x, y)) from the teacher LLM's output o (plus the input s in some cases), and p_T represents the teacher LLM with parameters θ_T .

Distillation pipeline in LLM era

Given the datasets $\mathcal{D}_I^{(\text{kd})}$ built for distillation, we then define a learning objective as

$$\mathcal{L} = \sum_I \mathcal{L}_I \left(\mathcal{D}_I^{(\text{kd})}; \theta_S \right)$$

where $\sum_I$ denotes there could be multiple tasks or skills being distilled into one student model, $\mathcal{L}_I(\cdot; \cdot)$ stands for a specific learning objective, and θ_S parameterizes the student model.

An overview of KD of LLM

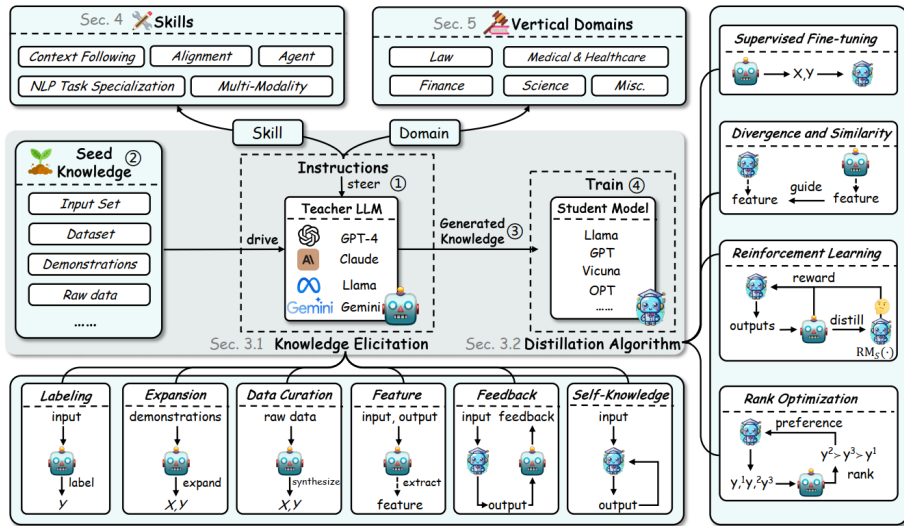


Table of Contents

- 1 Distilling the Knowledge in a Neural Network
- 2 Knowledge Distillation of Large Language Models
 - Introduction
 - Knowledge Distillation Algorithms
 - Skill Distillation
 - Domain-Specified Vertical Distillation
- 3 Examples: DistilBERT and TinyBERT
- 4 References

Labeling

- Labeling knowledge refers to using a teacher LLM to label the output y for a given input x as the seed knowledge, according to the instruction I or demonstrations c , where $c = (x_1, y_1), \dots, (x_n, y_n)$.
- It requires only the collection of an input dataset and feeding it into LLMs to obtain the desired generations. This process can be formulated as follows:

$$\mathcal{D}^{(\text{lab})} = \{x, y \mid x \sim \mathcal{X}, y \sim p_T(y \mid I \oplus c \oplus x)\}$$

Expansion

- While the labeling approach is simple and effective, it is constrained by the scale and variety of the input data.
- A key characteristic of expansion methods is the utilization of the in-context learning ability of LLMs to generate data similar to the provided demonstrations c . Unlike in the labeling approach, where the input x is sampled from the existing dataset, in the expansion approach, both x and y are generated by teacher LLMs. This process can be formulated as follows:

$$\mathcal{D}^{(\text{exp})} = \{(x, y) \mid x \sim p_T(x \mid I \oplus c), y \sim p_T(y \mid I \oplus x)\}.$$

Data Curation

- The pursuit of high-quality and scalable data generation in knowledge distillation from LLMs has led to the emergence of the Data Curation approach.
- Numerous diverse meta-information, such as topics or knowledge points, could be incorporated into this process to generate controllable x and y . Thus, this process can be meticulously controlled to yield datasets that are not only large in scale but also of high quality. The formulation for Data Curation can be represented as:

$$\mathcal{D}^{(\text{cur})} = \{(x, y) \mid x \sim p_T(x \mid I \oplus m), y \sim p_T(y \mid I \oplus x)\}$$

In this formulation, m represents the diverse meta-information used to guide the synthesis of x , and I is the instruction guiding teacher LLMs to generate x or y .

Feature

- The previously discussed knowledge elicitation methods are typically applied to powerful black-box models, which are expensive and somewhat unreproducible due to calling API. In contrast, white-box distillation offers a more transparent and accessible approach for researchers.
- The typical method for acquiring this feature knowledge involves teacher LLMs annotating the output sequence y with its internal representations. These annotations are then distilled into the student model using methods such as Kullback-Leibler Divergence (KLD). The process of eliciting feature knowledge can be formulated as follows:

$$\mathcal{D}^{(\text{feat})} = \{(x, y, \phi_{\text{feat}}(x, y; \theta_T)) \mid x \sim \mathcal{X}, y \sim \mathcal{Y}\}$$

- Most previous works predominantly focus on one-way knowledge transfer from the teacher to the student for imitation, without considering feedback from the teacher on the student's generation.
- The feedback from the teacher typically offers guidance on student-generated outputs by providing preferences, assessments, or corrective information. Here is a generalized formulation for eliciting feedback knowledge:

$$\mathcal{D}^{(\text{fb})} = \{(x, y, \phi_{\text{fb}}(x, y; \theta_T)) \mid x \sim \mathcal{X}, y \sim p_S(y \mid x)\}$$

where y denotes the output generated by the student model in response to x , and $\phi_{\text{fb}}(\cdot; \theta_T)$ represents providing feedback from teacher LLMs.

Self-Knowledge

- The knowledge could also be elicited from the student itself, which we refer to as Self-Knowledge. In this setting, the same model acts both as the teacher and the student, iteratively improving itself by distilling and refining its own previously generated outputs.
- This knowledge uniquely circumvents the need for an external, potentially proprietary, powerful teacher model, such as GPT-series LLMs. Eliciting self-knowledge could be formulated as:

$$\mathcal{D}^{(\text{sk})} = \{(x, y, \phi_{\text{sk}}(x, y)) \mid x \sim \mathcal{S}, y \sim p_S(y \mid I \oplus x)\}$$

where $\phi_{\text{sk}}(\cdot)$ is a generalized function that represents an additional process to the self-generated outputs y .

Summary

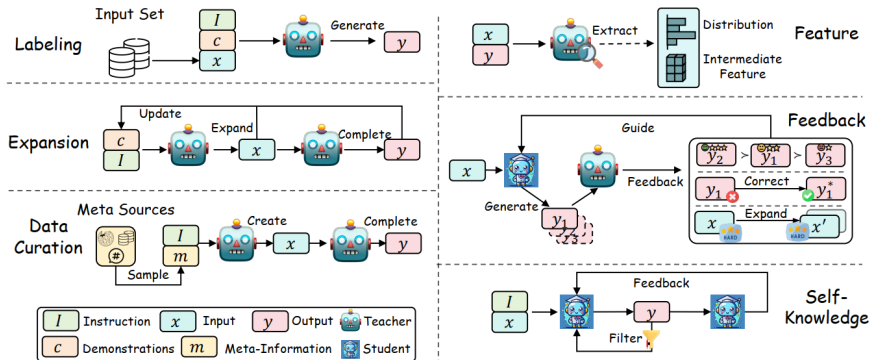


Fig. 5: An illustration of different knowledge elicitation methods from teacher LLMs. *Labeling*: The teacher generates the output from the input; *Expansion*: The teacher generates samples similar to the given demonstrations through in-context learning; *Data Curation*: The teacher synthesizes data according to meta-information, such as a topic or an entity; *Feature*: Feed the data into the teacher and extract its internal knowledge, such as logits and features; *Feedback*: The teacher provides feedback on the student's generations, such as preferences, corrections, expansions of challenging samples, etc; *Self-Knowledge*: The student first generates outputs, which is then filtered for high quality or evaluated by the student itself.

Supervised Fine-Tuning

- Supervised Fine-Tuning (SFT), or called Sequence-Level KD (SeqKD), is the simplest and one of the most effective methods for distilling powerful black-box LLMs.
- SFT fine-tunes student model by maximizing the likelihood of sequences generated by the teacher LLMs, aligning the student's predictions with those of the teacher. This process can be mathematically formulated as minimizing the objective function:

$$\mathcal{L}_{\text{SFT}} = \mathbb{E}_{x \sim \mathcal{X}, y \sim p_T(y|x)} [-\log p_S(y | x)],$$

where y is the output sequence produced by the teacher model.

Divergence and Similarity (White-box distillation)

- This section mainly concentrates on algorithms designed for distilling feature knowledge from white-box teacher LLMs, including distributions and hidden state features.
- These algorithms can be broadly categorized into two groups: those minimizing divergence in probability distributions and those aimed at enhancing the similarity of hidden states.

Divergence

Divergence-based methods minimize divergence between the probability distributions of the teacher and student models, represented by a general divergence function D :

$$L_{\text{Div}} = \mathbb{E}_{x \sim \mathcal{X}, y \sim \mathcal{Y}} [D(p_T(y | x), p_S(y | x))]$$

The specific form of D varies depending on the type of divergence employed.

Divergence Type	$D(p, q)$ Function
Forward KLD	$\sum p(t) \log \frac{p(t)}{q(t)}$
Reverse KLD	$\sum q(t) \log \frac{q(t)}{p(t)}$
JS Divergence	$\frac{1}{2} \left(\sum p(t) \log \frac{2p(t)}{p(t)+q(t)} + \sum q(t) \log \frac{2q(t)}{p(t)+q(t)} \right)$

Similarity

Similarity-based methods in knowledge distillation aim to align the hidden states or features of the student model with those of the teacher. The objective is to ensure that the student model not only produces similar outputs to the teacher but also processes information in a comparable manner.

$$\mathcal{L}_{\text{Sim}} = \mathbb{E}_{x \sim \mathcal{X}, y \sim \mathcal{Y}} [\mathcal{L}_F (\Phi_T (f_T(x, y)), \Phi_S (f_S(x, y)))]$$

where $f_T(x, y)$ and $f_S(x, y)$ are the feature maps of the teacher and student models, respectively.

Similarity Function $\mathcal{L}_F$	Expression
L2-Norm Distance	$\ \Phi_T (f_T(x, y)) - \Phi_S (f_S(x, y))\ _2$
L1-Norm Distance	$\ \Phi_T (f_T(x, y)) - \Phi_S (f_S(x, y))\ _1$
Cross-Entropy Loss	$-\sum \Phi_T (f_T(x, y)) \log (\Phi_S (f_S(x, y)))$
Maximum Mean Discrepancy	$\text{MMD} (\Phi_T (f_T(x, y)), \Phi_S (f_S(x, y)))$

Reinforcement Learning

This approach is especially relevant for leveraging the feedback from teacher to train student models. The RL-based distillation process typically involves two main stages:

- **Distilled Reward Model Training.** The first stage involves training a reward model r_ϕ using the feedback data $\mathcal{D}^{(\text{fd})}$ generated by teacher LLMs. The loss function for the reward model is defined as:

$$\mathcal{L}_{\text{RM}} \left(r_\phi, \mathcal{D}^{(\text{fd})} \right) = - \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}^{(\text{fd})}} \left[\log \sigma \left(r_\phi(x, y_w) - r_\phi(x, y_l) \right) \right]$$

Reinforcement Learning

- **Reinforcement Learning Optimization.** In the second stage, the student model, represented by a policy π_θ , is optimized to maximize the expected reward as per the trained reward model. Simultaneously, it minimizes the divergence from a reference policy π_{ref} , typically the initial policy of the student model trained by SFT, controlled by a factor β . The RL objective is given by:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim X, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta D_{\text{KL}} [\pi_\theta(y | x) \| \pi_{\text{ref}}(y | x)]$$

Table of Contents

- 1 Distilling the Knowledge in a Neural Network
- 2 Knowledge Distillation of Large Language Models
 - Introduction
 - Knowledge Distillation Algorithms
 - **Skill Distillation**
 - Domain-Specified Vertical Distillation
- 3 Examples: DistilBERT and TinyBERT
- 4 References

Context Following:

- Instruction following
 - ▶ Basic instructions
 - ▶ Complex instructions
 - ▶ Human instructions
 - ▶ System instructions
 - ▶ High-quality instructions
 - ▶ Improved instructions
- Multi-turn dialogue
- Retrieval-Augmented Generation (RAG) capability

Alignment

- Thinking pattern
- Preference
- Value

Agent

- Tool using
- Planning

NLP Task Specialization

- Natural language understanding
- Natural language generation
- Information retrieval
- Recommendation
- Text generation evaluation
- Code

Multi-Modality

Table of Contents

- 1 Distilling the Knowledge in a Neural Network
- 2 Knowledge Distillation of Large Language Models
 - Introduction
 - Knowledge Distillation Algorithms
 - Skill Distillation
 - Domain-Specified Vertical Distillation
- 3 Examples: DistilBERT and TinyBERT
- 4 References

Domain-Specified Vertical Distillation

This section shifts from skill distillation to examine KD of LLMs in various vertical domains, including Law, Medical & Healthcare, Finance, and Science, etc. It delves into customizing distilled LLMs for these fields, showing its significant role in enhancing domain-specific AI applications.

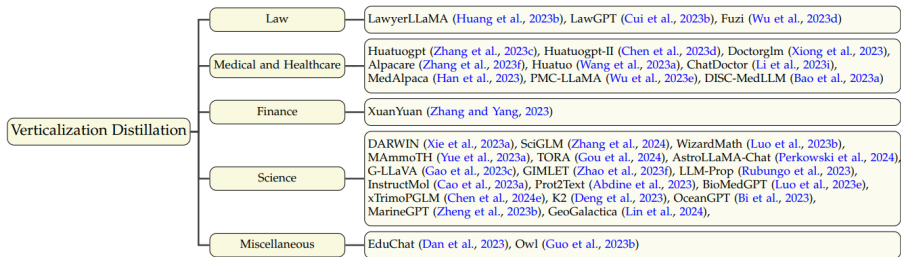


Fig. 7: Taxonomy of Verticalization Distillation.

Table of Contents

- 1 Distilling the Knowledge in a Neural Network
- 2 Knowledge Distillation of Large Language Models
 - Introduction
 - Knowledge Distillation Algorithms
 - Skill Distillation
 - Domain-Specified Vertical Distillation
- 3 Examples: DistilBERT and TinyBERT
- 4 References

DistilBERT: a distilled version of BERT

- **Student architecture:** In the present work, the student - DistilBERT - has the same general architecture as BERT. The token-type embeddings and the pooler are removed while the number of layers is reduced by a factor of 2.
- **Student initialization:** In addition to the previously described optimization and architectural choices, an important element in training procedure is to find the right initialization for the sub-network to converge. Taking advantage of the common dimensionality between teacher and student networks, we initialize the student from the teacher by taking one layer out of two.

DistilBERT: a distilled version of BERT

Training loss:

- The student is trained with a distillation loss over the soft target probabilities of the teacher: $L_{ce} = \sum_i t_i * \log(s_i)$ where t_i (resp. s_i) is a probability estimated by the teacher (resp. the student). This objective results in a rich training signal by leveraging the full teacher distribution. Following Hinton, we used a softmax-temperature: $p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$ where T controls the smoothness of the output distribution and z_i is the model score for the class i .
- The final training objective is a linear combination of the distillation loss L_{ce} with the supervised training loss, in our case the masked language modeling loss L_{mlm} .
- It's beneficial to add a cosine embedding loss (L_{cos}) which will tend to align the directions of the student and teacher hidden states vectors.

TinyBERT

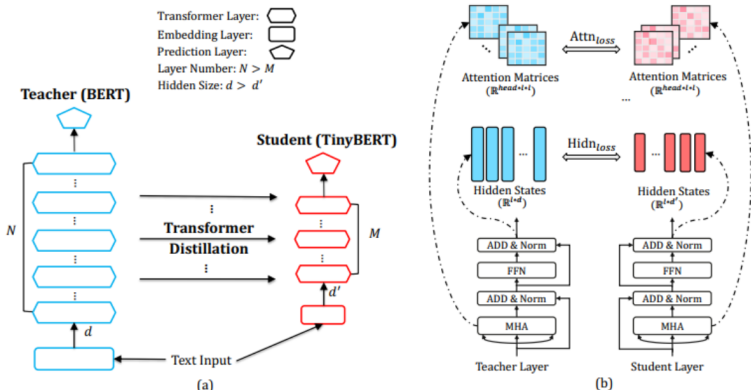


Figure 1: An overview of Transformer distillation: (a) the framework of Transformer distillation, (b) the details of Transformer-layer distillation consisting of Attn_{loss} (attention based distillation) and Hidn_{loss} (hidden states based distillation).

Training loss:

- **Transformer-layer Distillation:** The proposed Transformer-layer distillation includes the attention-based distillation and hidden-states based distillation:
 - ▶ Attention-based distillation: Specifically, the student learns to fit the matrices of multi-head attention in the teacher network, and the objective is defined as:

$$\mathcal{L}_{\text{attm}} = \frac{1}{h} \sum_{i=1}^h \text{MSE} \left(A_i^S, A_i^T \right)$$

- ▶ In addition to the attention based distillation, we also distill the knowledge from the output of Transformer layer, and the objective is as follows:

$$\mathcal{L}_{\text{hidn}} = \text{MSE} \left(H^S W_h, H^T \right),$$

Training loss:

- **Embedding-layer Distillation:** Similar to the hidden states based distillation, we also perform embedding-layer distillation and the objective is:

$$\mathcal{L}_{\text{embd}} = \text{MSE} \left(E^S W_e, E^T \right)$$

- **Prediction-layer Distillation:** We penalize the soft cross-entropy loss between the student network's logits against the teacher's logits:

$$\mathcal{L}_{\text{pred}} = \text{CE} \left(z^T / t, z^S / t \right)$$

Two-stage distillation:

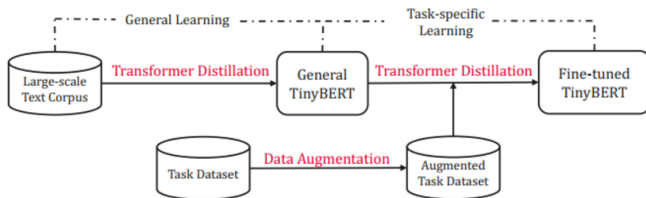


Figure 2: The illustration of TinyBERT learning

Table of Contents

- 1 Distilling the Knowledge in a Neural Network
- 2 Knowledge Distillation of Large Language Models
 - Introduction
 - Knowledge Distillation Algorithms
 - Skill Distillation
 - Domain-Specified Vertical Distillation
- 3 Examples: DistilBERT and TinyBERT
- 4 References

References

- 1 Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. "Distilling the knowledge in a neural network." arXiv preprint arXiv:1503.02531 (2015).
- 2 Xu, Xiaohan, et al. "A survey on knowledge distillation of large language models." arXiv preprint arXiv:2402.13116 (2024).
- 3 Jiao, Xiaoqi, et al. "Tinybert: Distilling bert for natural language understanding." arXiv preprint arXiv:1909.10351 (2019).
- 4 Sanh, Victor, et al. "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter." arXiv preprint arXiv:1910.01108 (2019).